

SQL Injection Attack — Listing Database Contents on Non-Oracle Databases

Platform: PortSwigger Web Security Academy

Lab: SQL injection attack, listing the database contents on non-Oracle databases

Vulnerability: SQL Injection

Database: Non-Oracle

Technique: UNION-Based SQL Injection

Testing Type: Authorized Security Testing

1. Overview

A SQL Injection vulnerability was successfully exploited in the PortSwigger Web Security Academy lab.

The vulnerability allowed manipulation of a backend SQL query through a user-controlled parameter. A UNION-based SQL Injection technique was used to enumerate the database structure and retrieve its contents.

2. Methodology

The assessment followed these steps:

1. Identified the vulnerable input parameter.
 2. Determined the number of columns returned by the original query.
 3. Identified columns capable of displaying text.
 4. Used database metadata to enumerate available tables.
 5. Identified the relevant application table and its columns.
 6. Retrieved the required database contents.
 7. Verified successful exploitation within the lab.
-

3. Technical Analysis

The vulnerability occurred because user-controlled input was incorporated into an SQL query without proper parameterization.

A UNION-based SQL Injection technique allowed the original query to be extended with an additional SQL query, making it possible to retrieve information from the underlying database.

For non-Oracle databases, metadata can commonly be accessed through structures such as `INFORMATION_SCHEMA` or database-specific system catalogs. The exact syntax depends on the database management system.

Attack flow:

Injection Point → Column Enumeration → Metadata Enumeration → Table Discovery → Data Retrieval

4. Impact

Successful exploitation could allow an attacker to:

- Enumerate database tables and columns.
- Discover the application's database structure.
- Access sensitive application data.
- Retrieve authentication-related information.
- Potentially compromise user accounts.
- Modify or delete data if sufficient database privileges are available.

Severity: High

5. Root Cause

The root cause was improper handling of user-controlled input within the SQL query.

The application failed to properly separate SQL commands from user-supplied data, allowing the query structure to be manipulated.

6. OWASP & CWE

OWASP Top 10: A03:2021 — Injection

CWE-89: Improper Neutralization of Special Elements used in an SQL Command (SQL Injection)

7. Remediation

- Use prepared statements and parameterized queries.
 - Never concatenate untrusted input directly into SQL queries.
 - Implement server-side input validation.
 - Apply least-privilege permissions to database accounts.
 - Prevent detailed database errors from being exposed.
 - Perform regular security testing.
-

8. Conclusion

The non-Oracle database enumeration lab was successfully completed. The exercise demonstrated how SQL Injection can be used to discover database structures and retrieve application data.

The primary defense is **parameterized queries**, supported by proper input validation, least-privilege database permissions, and secure error handling.

Evidence: Lab successfully completed; screenshots were not included.